

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 6
з курсу “Інформаційна безпека програм та даних”
на тему: “Виявлення вразливості SQL-ін’єкцій”

Виконав:

Студент. гр. ФєП-41 Фіняк Олег

Перевірив:

Павлик М.Р.

Львів – 2026

Мета: створити простий сценарій Python для виявлення вразливості SQL-ін'єкцій у веб-додатках.

Завдання:

1. Встановіть необхідні бібліотеки: `pip3 install requests bs4`.
2. Створіть новий файл `sql_injection_detector.py`, імпортуйте необхідні модулі, ініціалізуйте сеанс запитів і встановіть агент користувача:
3. Напишіть функцію `get_all_forms()` яка використовує бібліотеку BeautifulSoup для отримання всіх тегів форми з HTML і повертає їх як список Python:
4. Напишіть функцію `get_form_details()` яка отримує єдиний об'єкт тегу форми як аргумент і аналізує корисну інформацію про форму, таку як дія (цільова URL-адреса), метод (GET, POST тощо) і всі атрибути поля введення (тип, ім'я та значення).
5. Визначаємо функцію, яка повідомляє нам, чи є на веб-сторінці помилки SQL, це буде зручно під час перевірки вразливості SQL-ін'єкції
6. Визначаємо основну функцію, яка шукає всі форми на веб-сторінці та намагається розмістити символи лапок і подвійних лапок у полях введення. Перш ніж видобувати форми та надсилати їх, наведена вище функція спочатку перевіряє URL-адресу на наявність уразливості, оскільки сама URL-адреса може бути вразливою. Це робиться просто додаванням символу лапки до URL-адреси. Потім ми робимо запит за допомогою бібліотеки запитів і перевіряємо, чи вміст відповіді містить помилки, які ми шукаємо. Після цього ми аналізуємо форми та робимо надсилання із символами лапок у кожній знайденій формі.
7. Викликаємо основну функцію
8. Запускаємо скрипт командою: `python3 sql_injection_detector.py http://testphp.vulnweb.com/artists.php?artist=1`
9. Спробуйте виявити вразливість SQL-ін'єкцій у інших веб-додатках. Запишіть результати роботи `sql_injection_detector.py`.

```
PS C:\Users\Finik\Desktop\uni_labs_4\sec_sem\security\Lab_6> python sql_injection_detector.py http://demo.testfire.net/
[!] Trying http://demo.testfire.net/"
[!] Trying http://demo.testfire.net/'
[+] Detected 1 forms on http://demo.testfire.net/.
```

sql_injection_detector.py

```
import requests #
from bs4 import BeautifulSoup as bs #
from urllib.parse import urljoin #
from pprint import pprint #
import sys #
```

```
# Ініціалізуємо сеанс запитів і встановлюємо User-Agent #[cite: 6]
s = requests.Session() #[cite: 6]
s.headers["User-Agent"] = "Chrome/83.0.4103.106" #[cite: 6]

def get_all_forms(url): #[cite: 6]
    """Отримує всі मेзу form з HTML-сторінки.""" #[cite: 6]
    soup = bs(s.get(url).content, "html.parser") #[cite: 6]
    return soup.find_all("form") #[cite: 6]

def get_form_details(form): #[cite: 6]
    """Аналізує корисну інформацію про форму (action, method, inputs).""" #[cite:
6]
    details = {} #[cite: 6]

    # Отримуємо цільову URL-адресу форми (action) #[cite: 6]
    try:
        action = form.attrs.get("action").lower() #[cite: 6]
    except:
        action = None #[cite: 6]

    # Отримуємо метод форми (POST, GET тощо) #[cite: 6]
    method = form.attrs.get("method", "get").lower() #[cite: 6]

    # Отримуємо всі атрибути поля введення (type, name, value) #[cite: 6]
    inputs = [] #[cite: 6]
    for input_tag in form.find_all("input"): #[cite: 6]
        input_type = input_tag.attrs.get("type", "text") #[cite: 6]
        input_name = input_tag.attrs.get("name") #[cite: 6]
        input_value = input_tag.attrs.get("value", "") #[cite: 6]
        inputs.append({"type": input_type, "name": input_name, "value":
input_value}) #[cite: 6]

    # Додаємо все у словник #[cite: 6]
    details["action"] = action #[cite: 6]
    details["method"] = method #[cite: 6]
    details["inputs"] = inputs #[cite: 6]

    return details #[cite: 6]

def is_vulnerable(response): #[cite: 6]
    """Перевіряє, чи є на веб-сторінці помилки SQL у відповіді.""" #[cite: 6]
    errors = {
        # MySQL #[cite: 6]
        "you have an error in your sql syntax;", #[cite: 6]
        "warning: mysql", #[cite: 6]
        # SQL Server #[cite: 6]
        "unclosed quotation mark after the character string", #[cite: 6]
        # Oracle #[cite: 6]
        "quoted string not properly terminated", #[cite: 6]
    }

    for error in errors: #[cite: 6]
```

```

        # Якщо знайдено хоча б одну з помилок, повертаємо True #[cite: 6]
        if error in response.content.decode().lower(): #[cite: 6]
            return True #[cite: 6]

# Помилки не виявлено #[cite: 6]
return False #[cite: 6]

def scan_sql_injection(url): #[cite: 6]
    """Шукає форми та намагається розмістити ланки у полях введення і URL."""
#[cite: 6]
    # Перевірка самої URL-адреси #[cite: 6]
    for c in "\"'": #[cite: 6]
        # Додаємо символ ланки до URL-адреси #[cite: 6]
        new_url = f"{url}{c}" #[cite: 6]
        print("[!] Trying", new_url) #[cite: 6]

        # Робимо HTTP запит #[cite: 6]
        res = s.get(new_url) #[cite: 6]

        if is_vulnerable(res): #[cite: 6]
            print("[+] SQL Injection vulnerability detected, link:", new_url)
#[cite: 6]
            return #[cite: 6]

    # Перевірка HTML-форм #[cite: 6]
    forms = get_all_forms(url) #[cite: 6]
    print(f"[+] Detected {len(forms)} forms on {url}.") #[cite: 6]

    for form in forms: #[cite: 6]
        form_details = get_form_details(form) #[cite: 6]

        for c in "\"'": #[cite: 6]
            data = {} #[cite: 6]

            for input_tag in form_details["inputs"]: #[cite: 6]
                if input_tag["value"] or input_tag["type"] == "hidden": #[cite: 6]
                    try:
                        data[input_tag["name"]] = input_tag["value"] + c #[cite: 6]
                    except:
                        pass #[cite: 6]
                elif input_tag["type"] != "submit": #[cite: 6]
                    data[input_tag["name"]] = f"test{c}" #[cite: 6]

            # Об'єднуємо URL-адресу з action (URL-адресою запиту форми) #[cite: 6]
            form_url = urljoin(url, form_details["action"]) #[cite: 6]

            if form_details["method"] == "post": #[cite: 6]
                res = s.post(form_url, data=data) #[cite: 6]
            elif form_details["method"] == "get": #[cite: 6]
                res = s.get(form_url, params=data) #[cite: 6]

            # Перевіряємо, чи вразлива результуюча сторінка #[cite: 6]
            if is_vulnerable(res): #[cite: 6]

```

```

        print("[+] SQL Injection vulnerability detected, link:", form_url)
#[cite: 6]
        print("[+] Form:") #[cite: 6]
        pprint(form_details) #[cite: 6]
        break #[cite: 6]

if __name__ == "__main__": #[cite: 6]
    if len(sys.argv) > 1:
        url = sys.argv[1] #[cite: 6]
        scan_sql_injection(url) #[cite: 6]
    else:
        print("✗ Помилка: Вкажіть URL для перевірки.")
        print("Використання: python3 sql_injection_detector.py <URL>")

```

Висновок: Під час виконання лабораторної роботи було розроблено скрипт мовою Python для автоматизованого пошуку вразливостей SQL-ін'єкцій у веб-додатках. Практично закріплено навички парсингу HTML-форм за допомогою BeautifulSoup та масового виконання HTTP-запитів, що дозволяє тестувати реакцію бази даних сервера на впровадження спеціальних символів.